

GDB Playground — Exercises

A guided walk-through of `gdb_playground.c`. Three bugs of increasing subtlety, each chosen to exercise a different GDB technique.

Setup

Compile with maximum debug info, no optimization:

```
gcc -O0 -g3 -Wall -Wextra -o gdb_playground gdb_playground.c
```

Sanity check that the build works:

```
./gdb_playground 1.0 2.0 3.0
```

You should see a three-node sorted list, `sum = 6.0`, `max = 3.0`, and `nodes in [0.0, 5.0]: 3`.

Throughout the exercises we will keep a GDB session open:

```
gdb --args ./gdb_playground 1.0 2.0 3.0
```

If you are completely new to GDB, work through **Warm-up** first.

Otherwise jump to **Exercise 1**.

Warm-up — orient yourself

The exercises below practice the GDB workflow without trying to find any bug; the inputs we use here do not trigger the planted bugs.

1. List the main function.

```
(gdb) list main
```

`list` accepts a function name, a line number, or `file:line`.

2. Set a breakpoint at the start of main and run.

```
(gdb) break main
(gdb) run
```

Execution stops at the first executable line of `main`.

3. Step through line by line.

```
(gdb) next      # execute next line; do NOT enter functions
(gdb) next
(gdb) step      # execute next line; DO enter functions
```

4. Inspect simple variables.

```
(gdb) print argc
(gdb) print argv[1]
(gdb) print *(argv+1)
```

5. Inspect a struct once `head` exists.

Step until you have built at least one node, then:

```
(gdb) print *head
(gdb) print head->value
(gdb) print head->next
```

6. Continue and finish.

```
(gdb) continue    # run until next breakpoint or program end
(gdb) finish      # run until the current function returns
```

7. Backtrace.

```
(gdb) backtrace   # also "bt"
(gdb) bt full     # also prints locals in every frame
```

Now continue to Exercise 1.

Exercise 1 — the missing 5

Symptom. Run with five values:

```
./gdb_playground 1.0 2.0 3.0 4.0 5.0
```

The output ends with

```
nodes in [0.0, 5.0]: 4
```

But by inspection, all five values are inside `[0.0, 5.0]`. Why does the program report 4?

Hint. The function responsible is `count_in_range`. Set a breakpoint inside it and observe the comparison directly.

Method.

```
(gdb) break count_in_range
(gdb) run 1.0 2.0 3.0 4.0 5.0
(gdb) print lo
(gdb) print hi
```

Walk through the loop, watching the value at each step:

```
(gdb) next
(gdb) print c->value
(gdb) print (c->value >= lo)
(gdb) print (c->value < hi)
```

When `c->value` becomes 5.0, the second condition is false. Why?

The cause. The interval was advertised as closed `[lo, hi]`. The comparison uses `<` instead of `<=`. This is a classic half-open / closed convention confusion.

Fix. Change `<` to `<=` in `count_in_range`. Rebuild & run.

You should now see `nodes in [0.0, 5.0]: 5`.

Off-by-one bugs at interval boundaries are the most common kind of logic error in C. They are also the easiest to find with GDB, because the failing case is right there in the loop body and `print` reveals it immediately.

Exercise 2 — `find_kth` returns the wrong node

Symptom. Run with six values:

```
./gdb_playground 1.0 2.0 3.0 4.0 5.0 6.0
```

The output reports

```
k-th element (k=2): id=2 value=2.000
```

But the third element (`k=2`) of the sorted list `1.0 2.0 3.0 4.0 5.0 6.0` is `3.0`, not `2.0`. We are off by one.

Hint. Do not breakpoint at the function entry only. Use a **watchpoint** on the local variable `k` to see every time it is modified. You will need to enter the function first so that `k` exists as a watchable expression.

Method.

```
(gdb) break find_kth
(gdb) run 1.0 2.0 3.0 4.0 5.0 6.0
(gdb) watch k
(gdb) continue
```

GDB stops every time `k` changes. At each stop, inspect:

```
(gdb) print k
(gdb) print curr->id
(gdb) print curr->value
(gdb) continue
```

How many iterations did the loop actually take?

The Cause. The loop condition is `while ( --k > 0 && ... )`.

The `--k` is a **pre-decrement**: `k` is decremented *before* being compared to zero. For input `k=2`, the first iteration decrements `k` to 1 (true), advances `curr` once, then decrements to 0 (false) and exits. Net: only one advance, returning the second node instead of the third.

To advance two steps and return the third element, the condition must be `while ( k-- > 0 && ... )` (post-decrement: test, then decrement). Equivalently, restructure the loop to make the intent clearer.

Fix. Change `--k` to `k--`. Rebuild & run. You should now see `value=3.000`.

Pre-decrement versus post-decrement in loop conditions is a recurring source of off-by-one bugs. Whenever a loop terminates one step too early or one step too late, suspect a pre/post operator inversion.

Watchpoints on loop counters are the fastest way to catch this kind of bug; they let you see the variable change without manually stepping through every iteration.

Remind: GDB on x86_64 has only four hardware watchpoints (the debug registers DR0--DR3). Beyond that limit it falls back to software watchpoints, which step every instruction and slow execution by a lot. For a single loop counter the cost is negligible; for a large struct it is geological.

Exercise 3 — the max that is sometimes wrong

Symptom. Run with mixed signs:

```
./gdb_playground 1.0 -2.0 3.0
```

The output reports `max = 3.000`. Correct.

Now run with all negatives:

```
./gdb_playground -1.0 -2.0 -3.0
```

The output reports `max = 0.000`. But 0.0 is not in the list at all.

Hint. The function is recursive. Set a breakpoint at the function entry and use `backtrace` to see the entire recursion stack at the moment the base case returns. The bug lies in how the base case interacts with the recursive comparison.

Method.

```
(gdb) break find_max_recursive
(gdb) run -1.0 -2.0 -3.0
(gdb) continue          # let it recurse to the base case
(gdb) continue
(gdb) continue
(gdb) continue          # one more for the head==NULL call
```

At the `head == NULL` call:

```
(gdb) print head
(gdb) bt full
```

You should see four stacked calls to `find_max_recursive`. The innermost returned `0.0` (the base-case literal). The next-innermost compared its value (`-3.0`) to `0.0` and returned `0.0`. And so on, all the way up the chain.

To watch the propagation, finish each frame in turn and inspect the return value:

```
(gdb) finish      # return from the innermost call
(gdb) print rest_max
(gdb) print head->value
(gdb) finish
(gdb) print rest_max
...
```

The cause. The base case returns `0.0` for an empty list. This *looks* like a sensible default, but it pollutes the comparison at every level above. For any input where all values are negative, `head->value > 0.0` is always false, so `0.0` wins every comparison and propagates to the top.

The correct base case must return a value that *loses* every comparison against a real list value. Two options:

- Return `-INFINITY` (requires `#include <math.h>`); anything finite is greater than `-INFINITY`, so any real value wins.
- Special-case the single-node list as the base, so the recursion never sees an empty list:

```
if ( head->next == NULL ) return head->value;
```

Then handle the empty-list case at the caller and report "undefined" or similar.

The second option is structurally cleaner because it does not depend on floating-point semantics. The first is shorter.

Fix. Apply one of the two options. Rerun with all-negative input. You should now see the correct maximum.

Often recursive functions hide their bugs at the base case. The recursive step usually does the obvious thing; the base case quietly decides what "no input" means, and that decision can poison every result above it. When a recursive function returns plausible-but-wrong values on edge inputs, always inspect the base case first.